Ein eigenes Implementation Guide mit FSH

In diesem Lab erstellt ihr ein eigenes kleines Implementation Guide (IG) auf Basis von CH Core und veröffentlicht es auf GitHub Pages. Ihr setzt dabei alles aus der Vorlesung einmal selbst um: ein Profil, ein Binding auf Codes, eine vorgegebene Invariante und ein Beispiel.

Werkzeuge	eigener Laptop, GitHub-Account, Browser, ein Texteditor (z. B. VS Code)
Vorlage	`dg-ig.zip` auf Moodle
Abgabe	nur der Link zu eurem IG auf Moodle, Frist siehe Moodle

> Ihr müsst keinen IG Publisher installieren. Den vollständigen Build macht GitHub Actions bei jedem Push. Lokal braucht ihr nur SUSHI, um eure FSH-Dateien schnell zu prüfen.
>
> Wichtig ist, dass am Ende ein IG online ist, das ihr selbst gebaut habt.

Teil A: Vorbereitung

Ziel der Vorbereitung: Euer noch leeres IG ist unter eurer GitHub-Pages-Adresse online, und SUSHI läuft auf eurem Laptop.

Vorab: Werkzeuge installieren

Ihr braucht drei Werkzeuge. Prüft im Terminal mit dem Befehl in der rechten Spalte, ob sie schon installiert sind.

Werkzeug	Wofür	Installieren	Prüfen
Git	Vorlage auf GitHub hochladen	git-scm.com	`git --version`
Node.js (LTS)	wird von SUSHI gebraucht	nodejs.org	`node --version`
SUSHI	FSH kompilieren	`npm install -g fsh-sushi`	`sushi --version`

Schritt 1: Vorlage herunterladen und auf GitHub hochladen

Die Vorlage liegt als ZIP-Datei auf Moodle. Ihr legt auf GitHub ein leeres Repository an und ladet die Vorlage dort hoch.

1. `dg-ig.zip` von Moodle herunterladen und entpacken. Ihr erhaltet einen Ordner `dg-ig`. Er enthält auch die versteckten Einträge `.github` und `.gitignore`; ohne `.github` läuft später kein Build.
2. Auf GitHub oben rechts **+** → **New repository** wählen. Name: `dg-ig`. Sichtbarkeit: **Public**. GitHub Pages funktioniert mit einem kostenlosen Account nur für öffentliche Repositories. Kein README, keine .gitignore und keine Lizenz hinzufügen, das Repository muss leer sein.
3. Im Terminal in den Ordner wechseln und die Vorlage hochladen:

```
cd dg-ig
git init -b main
git add .
git commit -m "Vorlage"
git remote add origin https://github.com/<benutzername>/dg-ig.git
git push -u origin main
```

Schritt 2: GitHub Pages aktivieren

Direkt nach dem ersten Push läuft der Workflow schon einmal los und schlägt beim letzten Job fehl. Das ist normal, weil Pages noch nicht aktiv ist.

1. **Settings → Pages → Build and deployment → Source: GitHub Actions** wählen.
2. Tab **Actions** öffnen, den Lauf **IG bauen** anklicken, dann **Re-run all jobs**.
3. Warten, bis alle drei Jobs grün sind (1. SUSHI, 2. IG Publisher, 3. Veröffentlichen). Der IG Publisher braucht beim ersten Mal mehrere Minuten. In der Zwischenzeit könnt ihr mit Schritt 3 und 4 weitermachen.
4. Euer IG öffnen: `https://<benutzername>.github.io/dg-ig/`

Schritt 3: Konfiguration anpassen und SUSHI testen

Datei: `sushi-config.yaml`

Öffnet `sushi-config.yaml` und passt die Zeilen mit `TODO` an:

Feld	Beispiel
`id`	`ch.zhaw.dg.amuster`
`canonical`	`https://anna-muster.github.io/dg-ig`
`name`	`DGAMusterIG`
`title`	`"DG Lab IG Anna Muster: Arzneimittelallergie"`
`description`	ein Satz zu eurem Use Case
`publisher.name`	euer Name

Prüft danach, ob SUSHI mit eurer Vorlage läuft:

```
cd dg-ig
sushi build .
```

Beim ersten Lauf lädt SUSHI CH Core und CH Term herunter, das dauert etwas. Am Ende soll `0 Errors` stehen.

Teil B: Use Case Arzneimittelallergie

Eine Hausarztpraxis erfasst Arzneimittelallergien so, dass eine Notfallstation sie zuverlässig auswerten kann. Ressource: AllergyIntolerance, Parent: CH Core AllergyIntolerance.

Die Anforderungen kommen von einer Fachperson. Einige sind eindeutig, andere lassen bewusst mehrere Lösungen zu. Dort entscheidet ihr selbst. In Klammern steht das Element, um das es geht.

1. Das Profil ist nur für Allergien gegen Arzneimittel gedacht (`category`).
2. Der klinische Status muss immer angegeben sein (`clinicalStatus`).
3. Die Allergie wird mit Codes aus der Schweizer Allergieliste erfasst. Fehlt ein passender Code, soll trotzdem etwas erfasst werden können (`code`).
4. Für die Notfallstation ist wichtig, wie gefährlich die Allergie ist (`criticality`).
5. Der Beginn wird als Datum erfasst. Manche Patientinnen wissen aber nur noch ungefähr, in welchem Alter die Allergie aufgetreten ist (`onset[x]`).
6. Erfasst wird die Allergie von einer Ärztin oder einem Arzt (`recorder`).
7. Reaktionen sollen erfasst werden können, wenn sie bekannt sind (`reaction`, die Symptome stehen in `reaction.manifestation`).

Zusätzlich gilt eine Regel, die schon als Invariante in `invarianten.fsh` steht: Eine bestätigte Allergie muss mindestens eine Reaktion dokumentieren.

Aliase für Bindings im Profil

Element	Alias des ValueSets
`code`	`$ch-allergy-vs` (Schweizer Allergieliste)
`reaction.manifestation`	`$ch-allergy-manifestation-vs` (Symptome einer Reaktion)

Codes für das Beispiel (Schritt 6)

Element	Code
`category`	`#medication`
`clinicalStatus`	`$allergy-clinical#active`
`verificationStatus`	`$allergy-verification#confirmed`, `#unconfirmed`
`code`	`$sct#91936005 "Allergy to penicillin (finding)"`, `$sct#294499007 "Allergy to benzylpenicillin (finding)"`
`criticality`	`#low`, `#high`, `#unable-to-assess`
`onset[x]` als Alter	`* onsetAge = 30 'a' "Jahre"`
`reaction.manifestation`	`$sct#126485001 "Urticaria (disorder)"`

Teil C: Umsetzung

Ihr arbeitet nur in den Dateien unter `input/fsh/` und `input/pagecontent/`. Nach jeder grösseren Änderung lokal `sushi build .` ausführen.

Schritt 4: Anforderungen in Regeln übersetzen

Die Tabelle zeigt, welche Art von Regel zu welcher Formulierung meist passt.

Wenn die Anforderung sagt …	… dann ist das meist	FSH-Beispiel (aus anderen Ressourcen)
muss immer angegeben sein	Kardinalität	`* period 1..1`
darf höchstens einmal vorkommen	Kardinalität	`* reasonCode 0..1`
Systeme müssen es verarbeiten können	Must Support	`* serviceProvider MS`
nur Codes aus dieser Liste	Binding	`* priority from MeinValueSetVS (required)`
nur als Zeitpunkt, nicht als Zeitraum oder Text	Typ	`* effective[x] only dateTime`
ist immer dieser Wert	Wert fixieren	`* status = #final`
darf nur auf X verweisen	Referenz einschränken	`* requester only Reference(CHCorePractitionerRole)`

Bei einigen Anforderungen passen mehrere Regeln, z. B. `1..1` oder nur `MS`, `required` oder `extensible`. Dort gibt es keine einzig richtige Lösung. Entscheidend ist, dass eure Regel zur Anforderung passt.

Was CH Core schon festlegt, dürft ihr nur enger machen, nicht lockern. Bei Bindings heisst das: gleiche oder stärkere Stärke (example, preferred, extensible, required).

Schritt 5: Das Profil schreiben

Datei: `input/fsh/profil.fsh`

Die Kopfzeilen des Profils sind schon ausgefüllt. Für jede Anforderung steht eine auskommentierte Zeile mit dem Element:

```
// Anforderung 4: Kritikalität (wie gefährlich die Allergie ist)
// * criticality TODO_Regel
```

- Entfernt die `//` und ersetzt `TODO_Regel` durch eure Regel, z. B. eine Kardinalität oder `MS`.
- Hier stehen Regeln für alle Allergien. Einen Wert setzt ihr im Profil nur, wenn er immer gilt (Wert fixieren). Werte für eine bestimmte Patientin, z. B. `#high`, gehören ins Beispiel.
- Eine Anforderung kann mehrere Zeilen brauchen, z. B. eine Kardinalität und ein Binding für dasselbe Element.
- Bei `[x]`-Elementen schreibt ihr im Profil `onset[x]`, im Beispiel später `onsetDateTime` oder `onsetAge`.
- Am Ende entfernt ihr die `//` vor `* obeys dg-allergy-1`. Damit gilt die vorgegebene Invariante für euer Profil.

Schritt 6: Beispiel erstellen

Datei: `input/fsh/beispiele.fsh`

In der Datei steht ein Gerüst für ein Beispiel. Entfernt die `//` und ersetzt die TODOs. Patientin, Ärztin und Praxis sind in `basis-beispiele.fsh` schon da:

Instanz	Profil	Verweis im Beispiel
Anna Muster	CH Core Patient	`Reference(AnnaMuster)`
Dr. med. Sara Keller	CH Core Practitioner	`Reference(DrSaraKeller)`
Praxis am Park	CH Core Organization	`Reference(PraxisAmPark)`
Dr. Keller in der Praxis am Park	CH Core PractitionerRole	`Reference(DrSaraKellerPraxisAmPark)`

Das Beispiel muss alle Regeln eures Profils erfüllen. Wegen der Invariante braucht eine bestätigte Allergie (`confirmed`) eine Reaktion. SUSHI prüft das nicht, das zeigt erst der QA-Report.

Schritt 7: Startseite schreiben

Datei: `input/pagecontent/index.md`

Ersetzt die TODOs: Use Case in zwei bis drei Sätzen, Scope, eine kurze Liste eurer Artefakte und euer Name. Das ist das Erste, was man auf eurer Seite sieht.

Schritt 8: Pushen und den QA-Report lesen

Sobald `sushi build .` lokal `0 Errors` meldet:

```
git add .
git commit -m "Profil und Beispiel"
git push
```

Im Tab **Actions** seht ihr den Lauf. Unter dem Job **2. IG Publisher** steht eine Zusammenfassung mit der Anzahl Errors, Warnings und Hints. Wenn alle Jobs grün sind, öffnet `https://<benutzername>.github.io/dg-ig/qa.html`.

Meldung	Was ihr tut
Error	beheben
Warning	beheben oder in `input/ignoreWarnings.txt` eintragen
Hint	lesen, meist harmlos

Ein voller Build dauert mehrere Minuten. Pusht deshalb nicht nach jeder kleinen Änderung, sondern prüft lokal mit SUSHI und pusht, wenn ein Schritt fertig ist. Ein neuer Push bricht einen laufenden Build ab.

Teil D: Abgabe

Euer IG ist vollständig, wenn diese Punkte erfüllt sind:

- Das IG ist auf GitHub Pages erreichbar.
- `sushi-config.yaml` ist angepasst (kein `vorlage` mehr).
- Für jede der sieben Anforderungen steht im Profil mindestens eine Regel.
- Die Invariante ist im Profil mit `obeys` angewendet.
- Ein gültiges Beispiel zu eurem Profil.
- Die Startseite `index.md` ist ausgefüllt.
- Der QA-Report hat keine Errors.

Ihr gebt auf Moodle nur den Link zu eurem IG auf GitHub Pages ab, z. B. `https://anna-muster.github.io/dg-ig/`. Die Abgabefrist steht auf Moodle.